Syntax

– Terms refer to objects in a domain
– Predicates are used to make truth statements about objects

Functions

$\mathcal{F}$ is the set of all functions.
– A function f of arity n map n terms to one term
– Constants are functions of arity 0

Terms

– functions $f \in \mathcal{F}$ refer to objects implicitly through other terms
– constants $c \in \mathcal{F}$ refer to individual objects
– variables $v \in \mathcal{V}$ are placeholders for objects

The set of terms $\mathcal{T}$ is defined inductively:
– $\forall v \in V : v \in \mathcal{T}$
– $\forall c \in \mathcal{F} : c \in \mathcal{T}$
– $\forall f$ (of arity n) $\in \mathcal{F} : \forall t_1, \dots, t_n \in \mathcal{T} : f(t_1, \dots, t_n) \in \mathcal{T}$

Predicates

$\mathcal{P}$ refers to the set of predicate symbols
– A predicate symbol $P \in \mathcal{P}$ of arity n maps n terms to an atom.
– A predicate symbol $P \in \mathcal{P}$ of arity 0 acts like a propsitional variable.

Formulas

The set of formulas is inductively defined as:
– If φ is a formula, then $\neg\varphi$ is a formula
– If φ, ψ are formulas, then $\varphi \odot \psi$ is a formula where $\odot \in \{\vee, \wedge, \to\}$
– If φ is a formula, then $\forall v \in \mathcal{V} : \mathcal{Q}x : \psi$ is a formula where $\mathcal{Q} \in \{\exists, \forall\}$

Ground Terms/Formulas

A term/formula is ground if it contains no variable.

Subformulas

– If φ is a atomic, then
$$subformulas(\varphi) = \{\varphi\}$$

– If φ is of the form $\neg\psi, \forall x : \psi, \exists x : \psi,$ then
$$subformulas(\varphi) = \{\varphi\} \cup subformulas(\psi)$$

– If φ is of the from $\psi_1 \odot \psi_2$ where $\odot \in \{\vee, \wedge, \to\}$ then
$$subformulas(\varphi) = \{\varphi\} \cup subformulas(\psi_1) \cup subformulas(\psi_2)$$

A formula $\psi \in subformulas(\varphi)$ is called a proper subformula of φ, if $\varphi \neq \psi$.

Binding Strength (to avoid parentheses)

$1.\forall, \exists, \neg,\ 2.\wedge,\ 3.\vee,\ 4.\ \to$

– **Example** :
$$\neg A \wedge \exists x B \to C \vee D \leftrightarrow E$$
is read as: $(((\neg A) \wedge (\exists x B)) \to (C \vee D) \leftrightarrow E)$

Quantifier scope and bound variables

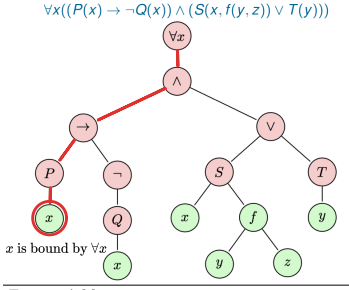
Given a formula $\mathcal{Q}x : \psi$ where $\mathcal{Q} \in \{\exists, \forall\}$:
– variable $x \in \mathcal{V}$ is bound in ψ
– ψ is the scope of x

The quantifier only binds the first occurrence of a variable if outside a parentheses.

– **Example:**
In the formula $\exists x P(x) \wedge R(x)$ only the first occurrence of x is bound, the second occurrence is free

Given a formula tree of a formula, a variable x is bound if it has an upward path to $\mathcal{Q}x$.

$\forall x((P(x) \to \neg Q(x)) \wedge (S(x, f(y, z)) \vee T(y)))$



x is bound by $\forall x$

Free variables

Given a term t, the set $free(t)$ of its free variables is defined as:
– If t is a variable $x \in \mathcal{V}$, then
$$free(t) = \{x\}$$

– If t is a constant, then
$$free(t) = \emptyset$$

– If t is of the form $f(t_1, \dots, t_n)$, then
$$free(t) = \bigcup_{i=1}^{n} free(t_i)$$

Given a formula φ, the set $free(\varphi)$ of its free variables is defined as:
– If φ is an atom $P(t_1, \dots, t_n)$, then
$$free(\varphi) = \bigcup_{i=1}^{n} free(t_i)$$

– If $\varphi = \neg\psi$, then
$$free(\varphi) = free(\psi)$$

– If $\varphi = \psi_1 \odot \psi_2$ where $\odot \in \{\vee, \wedge, \to\}$, then
$$free(\varphi) = free(\psi_1) \cup free(\psi_2)$$

– If $\varphi = \mathcal{Q}x : \psi$ where $\mathcal{Q} \in \{\forall, \exists\}$, then
$$free(\varphi) = free(\psi) \setminus \{x\}$$

Sentence

A formula is called a sentence if it contains no free variables.

Theory

A set $\Gamma = \{\varphi_1, \varphi_2, \dots\}$ of sentences is called a theory

Structures

A structure $\mathcal{M} = (\mathcal{D}, \mathcal{I})$ is made up of
– a domain $\mathcal{D}$
– an interpretation function $\mathcal{I}$
Constants, functions and predicates define the language (also called signature) $\mathcal{L}$ of the structure.

Domains

A domain $\mathcal{D}$ is a non-empty set of objects

Interpretation Function

The interpretation function $\mathcal{I}$ maps:
– each constant c to an object $I(c) = D$
– each function f of arity n to a total function $I(f) = D^n \to D$
– each predicate symbol P of arity n to a relation $I(P) \subseteq D^n$

One can also write $c^{\mathcal{M}}, f^{\mathcal{M}}, P^{\mathcal{M}}$ instead of $\mathcal{I}(\dots)$.

Satisfaction of ground formulas

For a structure $\mathcal{M} = (\mathcal{D}, \mathcal{I})$ and an atom $P(t_1, \dots, t_n)$ it holds:
– $\mathcal{M} \models P(t_1, \dots, t_n) \iff (t_1^{\mathcal{M}}, \dots, t_n^{\mathcal{M}}) \in P^{\mathcal{M}}$

Furthermore, for any ground formula φ it holds:
– $\mathcal{M} \models \neg\varphi \iff \neg(\mathcal{M} \models \varphi)$
– $\mathcal{M} \models \varphi \odot \psi \iff (\mathcal{M} \models \varphi) \odot (\mathcal{M} \models \psi)$ for $\odot \in \{\vee, \wedge, \to\}$

Given only a structure, one can only prove for ground formulas

Satisfaction of Formulas

Given an interpretation $(\mathcal{M}, \ell)$ the satisfaction of formulas is defined as:
– $(\mathcal{M}, \ell) \models P(t_1, \dots, t_n) \iff (t_1^{(\mathcal{M},\ell)}, \dots, t_n^{(\mathcal{M},\ell)})$
– $(\mathcal{M}, \ell) \models \neg\varphi \iff \neg((\mathcal{M}, \ell) \models \varphi)$
– $(\mathcal{M}, \ell) \models \varphi \odot \psi \iff ((\mathcal{M}, \ell) \models \varphi) \odot ((\mathcal{M}, \ell) \models \psi)$
– $(\mathcal{M}, \ell) \models \mathcal{Q}x : \varphi \iff \mathcal{Q}x : ((\mathcal{M}, \ell) \models \varphi)$ for $\mathcal{Q} \in \{\exists, \forall\}$

One may also write:
– $(\mathcal{M}, \ell) \not\models \varphi \iff \neg((\mathcal{M}, \ell) \models \varphi), (\mathcal{M}, \ell) \models \varphi \iff \mathcal{M} \models_\ell \varphi$

Satisfaction of Sentences

– **Theorem**
–Given a structure $\mathcal{M}$, a formula φ and variable assignments ℓ, ℓ', it holds:
$(\forall x \in free(\varphi) : \ell(x) = \ell'(x)) \implies (((\mathcal{M}, \ell) \models \varphi) \iff ((\mathcal{M}, \ell') \models \varphi))$

–Given a structure $\mathcal{M}$ and a sentence φ, it holds:
$\forall \ell, \ell' : ((\mathcal{M}, \ell) \models \varphi) \iff ((\mathcal{M}, \ell') \models \varphi)$

Satisfiability Definition

A formula φ is satisfiable $\iff \exists (\mathcal{M}, \ell) : (\mathcal{M}, \ell) \models \varphi$

Consistent Theory

A theory Γ is consistent $\iff \exists \mathcal{M} : \forall \varphi \in \Gamma : \mathcal{M} \models \varphi$

– **Theorem:** $(\forall \varphi \in \Gamma : \mathcal{M} \models \varphi) \iff \left(\mathcal{M} \models \bigwedge_{\varphi \in \Gamma} \varphi\right)$

Tableau

Same inference rules as PL tableau but with additional rules:

$$\frac{\mathbf{t} : \forall x F}{\mathbf{t} : F(t/x)} \qquad \frac{\mathbf{f} : \exists x F}{\mathbf{f} : F(t/x)} \qquad \frac{\mathbf{f} : \forall x F}{\mathbf{f} : F(c/x)} \qquad \frac{\mathbf{t} : \exists x F}{\mathbf{t} : F(c/x)}$$

For any variable-free term $t \in \Sigma^{\text{par}} \cup \Sigma$ (usually picking a term) already in tableau branch.

For a new Parameter $c \in \Sigma^{\text{par}}$ not occurring in the tableau branch

Note that for every ground-term in the branch the rule must be applied

Parameters are a set of (countably) infinitely many constants added to the language of $\mathcal{M}$

$F(t/x)$ denotes the formula F where all occurrences of x are replaced by t.

Undecidability of FOL (example)

Due to the $\mathbf{t} : \forall x F$ and $\mathbf{f} : \exists x F$ rules the tableau may expand forever, as can be seen with the following example.
A tableau is only guaranteed to terminate if the formula is a contradiction.

Given an interpretation (M, ℓ) and the formula: $\forall x : P(x, g(x))$, the tableau would expand as follows:
$\mathbf{t} : \forall x : P(x, g(x))$ //Applying the inference rule by replacing x for a term c
$\mathbf{t} : P(c, g(c))$ //since $g(c) = c'$ maps to a term, a new ground term has appeared in the branch for which this rule must be applied
$\mathbf{t} : P(c, g(c'))$ //once again, a new ground term $g(c') = c''$ has appeared in the branch. This will never stop

Example of a structure

– constants: $zero$
– functions: $succ$ (arity 1)
– predicates: $nonneg$ (arity 1), $equals$ (arity 2)

$\mathcal{D} = \{a, aa, aaa, \dots\}$
$zero^{\mathcal{M}} = a$
$\forall d \in \mathcal{D} : succ^{\mathcal{M}}(d) = da$
$nonneg^{\mathcal{M}} = \{aa, aaa, \dots\}$
$equals^{\mathcal{M}} = \{(d, d) \mid d \in \mathcal{D}\}$

The structure satisfies the following ground atom:

– $equals(aa, succ(a))$

since $succ(a) = aa$
and $(aa, aa) \in equals^{\mathcal{M}}$

FOL Interpretation

An interpretation is a pair $(\mathcal{M}, \ell)$.
An interpretation of a term $t^{(\mathcal{M},\ell)}$ is defined as:
– $t^{(\mathcal{M},\ell)} = \ell(t)$ if $t \in \mathcal{V}$
– $t^{(\mathcal{M},\ell)} = f^{\mathcal{M}}(t_1^{(\mathcal{M},\ell)}, \dots, t_n^{(\mathcal{M},\ell)})$ if $t = f(t_1, \dots, t_n)$

Modelling natural language

– All As are B
$\forall x : (A(x) \to B(x))$

– Some As are Bs
$\exists x : (A(x) \wedge B(x))$

– No As are B
$\neg\exists x : (A(x) \wedge B(x))$

– Some As are not Bs
$\exists x : (A(x) \wedge \neg B(x))$

(*Typically*) adjectives and verbs are predicates; nouns are terms

Herbrand Interpretation

A Herbrand interpretation is an interpretation which maps functions to applications of functions on terms
A Herbrand universe is the set of all ground terms.

– **Example:** (same idea as before)
$\mathcal{D} = \{\text{zero}, succ(\text{zero}), succ(succ(\text{zero})), \dots\}$
$zero^{\mathcal{M}} = \text{zero}$
$\dots$

Variable Assignment

Given a structure $\mathcal{M}$, for a variable assignment $\ell : \mathcal{V} \to \mathcal{D}$ holds:

– Given variable $x \in \mathcal{V}$ and object $d \in D$: $\ell[d/x]$ is a variable assignment such that
$$\ell[d/x](y) = \begin{cases} d & \text{if } y = x \\ \ell(y) & \text{if } y \neq x \end{cases}$$

Modelling natural language (Examples)

– One vehicle passed all inspections:
$\exists x : (\text{Vehicle}(x) \wedge \forall y : (\text{Inspection}(y) \to \text{Passed}(x, y)))$

– There is one vehicle in every lane:
$\forall x : (\text{Lane}(x) \to \exists y : (\text{Vehicle}(y) \wedge \text{InLane}(y, x)))$

– All the children eat a sandwich
$\forall x : (\text{Child}(x) \to \exists y : (\text{Sandwich}(y) \wedge \text{Eats}(x, y)))$

– All the children are in the playground:
$\exists y : (\text{Playground}(y) \wedge \forall x : (\text{Child}(x) \to \text{IsIn}(x, y)))$

The robot removes a block. Then it places a block on the table:
$\exists x_1 : (\text{Block}(x_1) \wedge \text{Removes}(robot, x_1)) \wedge$
$\exists x_2 : (\text{Block}(x_2) \wedge \text{PlacesOnTable}(robot, x_2))$

Equivalences of quantified formulas

$\neg\forall x : \varphi \equiv \exists x : \neg\varphi \qquad \neg\exists x : \varphi \equiv \forall x : \neg\varphi$
$\forall x : \varphi \wedge \forall x : \psi \equiv \forall x : (\varphi \wedge \psi) \qquad \exists x : \varphi \vee \exists x : \psi \equiv \exists x : (\varphi \vee \psi)$
$\mathcal{Q}x : \mathcal{Q}y : \varphi \equiv \mathcal{Q}y : \mathcal{Q}x : \varphi$

If x not free in ψ :
$\psi \odot \mathcal{Q}x : \varphi \equiv \mathcal{Q}x : (\varphi \odot \psi) \qquad$ for $\mathcal{Q} \in \{\forall, \exists\},\ \odot \in \{\vee, \wedge\}$

Negation Normal Form

A formula φ is in NNF if the only connectives are $\odot \in \{\vee, \wedge, \neg\}$ where $\neg$ is only used on variables

Models and Consequences

Each theory Γ has:
– Models: $Mods(\Gamma) = \{\mathcal{M} \mid \mathcal{M} \text{ is a structure}, \mathcal{M} \models \Gamma\}$
– Consequences: $Cons(\Gamma) = \{\varphi \mid \varphi \text{ is a sentence}, \Gamma \models \varphi\}$

– **Monotonicity Theorem:**
$Mods(\Gamma \cup \{\varphi\}) \subseteq Mods(\Gamma),\ Cons(\Gamma) \subseteq Cons(\Gamma \cup \{\varphi\})$
$(\Gamma \models \varphi) \implies (\forall \Gamma' : ((\Gamma \subseteq \Gamma') \implies (\Gamma' \models \varphi)))$

Formulas φ, ψ are equivalent $\varphi \equiv \psi \iff Mods(\varphi) = Mods(\psi)$

Prenex Normal Form

A formula $\varphi = \mathcal{Q}_1 x_1 : \dots \mathcal{Q}_n x_n : \psi$ is in PNF with:

– $\mathcal{Q}_i \in \{\forall, \exists\}$ for $i, \dots, n$ (called quantifier prefix)
– ψ is a quantifier-free formula (called matrix)

Computing PNF

1. Eliminate all occurrences of $\to$
2. Move all negations inwards
3. Rename all variables apart
4. Move quantifiers to the front

Example of computing the PNF

Given the formula: $\forall x : (\exists y : R(x, y) \wedge \forall y : \neg S(x, y) \to \neg(\exists y : R(x, y) \wedge P))$

– Step 1:
$\forall x : (\neg(\exists y : R(x, y) \wedge \forall y : S(x, y)) \vee \neg(\exists y : R(x, y) \wedge P))$

– Step 2:
$\forall x : (\forall y : \neg R(x, y) \vee \exists y : S(x, y) \vee \forall y : \neg R(x, y) \vee \neg P)$

– Step 3:
$\forall x : (\forall y_1 : \neg R(x, y_1) \vee \exists y_2 : S(x, y) \vee \forall y_3 : \neg R(x, y_3) \vee \neg P)$

– Step 4:
$\forall x : \forall y_1 : \exists y_2 : \forall y_3 : (\neg R(x, y_1) \vee S(x, y_2) \vee \neg R(x, y_3) \vee \neg P))$

FOL